Especificação Técnica Integral — Copilot Protheus

> **Finalidade:** documento de contexto operacional e técnico para colaboração entre equipes humanas e outras IAs no projeto Copilot Protheus.
> **Status:** arquitetura em evolução. Este documento separa explicitamente o estado legado/documentado do estado-alvo obrigatório da integração OAuth2 por tenant e ambiente.
> **Data de consolidação:** 19/08/2026

1. Objetivo do produto

O **Copilot Protheus** é uma plataforma corporativa de IA para empresas usuárias do TOTVS Protheus. Ela deve permitir consulta assistida, análise de dados, RAG e geração de SQL Oracle com governança granular por tenant, empresa, ambiente, papel, tabela e campo.

A plataforma opera em modelo **multi-tenant**, com isolamento de dados de dicionário e governança no PostgreSQL, e consulta dados operacionais no ERP Protheus através de APIs REST, especialmente `QueryRest`.

1.1 Princípios inegociáveis

1. **Zero alucinação de dados empresariais.** Se o Protheus não responder, a consulta retornar vazio ou a permissão não permitir a extração, o sistema deve informar o bloqueio com transparência. Nunca estimar valores financeiros, fiscais, contábeis ou operacionais.
2. **Isolamento multi-tenant.** Um usuário, tenant, empresa ou ambiente não pode acessar metadados ou dados de outro tenant.
3. **Menor privilégio.** A integração Protheus deve ser de leitura, com usuário técnico dedicado, permissões mínimas e controle por tabela/campo.
4. **Credenciais separadas.** Credencial do painel Copilot é diferente da credencial REST do Protheus. Elas nunca podem ser reutilizadas ou mapeadas entre si.
5. **Token, não Basic por consulta.** Chamadas operacionais ao QueryRest usam `Authorization: Bearer <access_token>`. Usuário e senha do Protheus são usados somente na emissão/renovação do token.
6. **Catálogo global separado de conexão.** O catálogo global de módulos não possui tenant, URL, empresa, senha ou token.
7. **Segurança por padrão.** Tokens, senhas e headers `Authorization` não aparecem em logs, respostas de API, auditoria ou interfaces.

2. Arquitetura de alto nível

```
graph TD
    U[Usuário final / analista] -->|HTTPS| CHAT[Frontend do chat\nCloudflare Pages]
    A[Administrador] -->|HTTPS| EDGE[Cloudflare Edge / Tunnel]
    CHAT -->|HTTPS API| EDGE
    EDGE -->|/admin| ADMIN[Admin Frontend\nReact/Vite/Nginx]
    EDGE -->|/api| API[Backend\nFastAPI/Uvicorn]
    API -->|SQL + vetores| PG[(PostgreSQL 16 + pgvector)]
    API -->|RAG| R2[Cloudflare R2]
    API -->|LLM| LLM[Gemini / OpenAI / Ollama]
    API -->|REST seguro| MW[Middleware Node/Express]
    MW -->|HTTPS/VPN| ERP[TOTVS Protheus\nAppServer REST / QueryRest]
```

2.1 Hospedagem e rede

Componente	Tecnologia	Localização	Responsabilidade
Frontend do chat	React + Vite	Cloudflare Pages	Interface final do usuário
Painel administrativo	React + Vite + Nginx	Hetzner, container Docker	Gestão de tenants, empresas, dicionário, permissões e operação
Backend	Python, FastAPI, Uvicorn, SQLAlchemy	Hetzner, container Docker	Regras de negócio, RAG, RBAC, APIs, sincronização e integração ERP
Middleware	Node.js, Express, Axios	Hetzner, rede Docker interna	Resiliência/proxy para integração Protheus
Banco	PostgreSQL 16 + pgvector	Hetzner, container Docker	Dados relacionais, governança, metadados e vetores
Tunnel	cloudflared	Hetzner, container Docker	Exposição segura sem portas públicas de aplicação
RAG storage	Cloudflare R2	Cloudflare	Arquivos e artefatos de conhecimento
LLM local	Ollama	Host Hetzner	Inferência local para cenários sensíveis
ERP	TOTVS Protheus / Oracle	Cliente	Dados operacionais e APIs REST

2.2 Endereços conhecidos

- Chat público: `https://copilot.elitecorp.tec.br`
- API/admin via Cloudflare: `https://copilot-api.elitecorp.tec.br`
- Administração: `https://copilot-api.elitecorp.tec.br/admin`
- Backend interno: porta `8000`
- Admin frontend interno: porta `5174`
- Middleware interno: porta `3001`
- PostgreSQL interno: porta `5432`; manutenção na VPS: porta mapeada `5435`
- Ollama a partir do Docker: `http://host.docker.internal:11434`

2.3 Operação de containers

Nomes observados/esperados:

- `copilot-protheus-backend`
- `copilot-protheus-admin-frontend`
- `copilot-protheus-frontend`
- `copilot-protheus-middleware`
- `copilot-protheus-db`
- `copilot-protheus-cloudflared`

Raiz operacional na VPS:

```
cd /root/copilotprotheus
```

3. Repositório e componentes

Estrutura de referência:

```
copilotprotheus/
├── .agents/                    # Instruções para agentes de desenvolvimento
├── admin-frontend/             # Painel administrativo React/Vite
├── backend/                    # FastAPI, serviços, modelos, schemas e migrations
│   ├── app/
│   │   ├── api/                # Rotas REST
│   │   ├── core/               # Segurança, settings, autenticação, logs
│   │   ├── db/                 # Engine, sessões, inicialização/migrations
│   │   ├── models/             # ORM SQLAlchemy
│   │   ├── schemas/            # Pydantic request/response
│   │   └── services/           # Regras de negócio e integrações
│   ├── scripts/
│   ├── Dockerfile
│   └── requirements.txt
├── database/                   # Migrations SQL, quando aplicável
├── db/                         # Configuração/artefatos do serviço PostgreSQL
├── docs/                       # Documentação e contratos
├── edge_extension/             # Extensão Chrome/WebApp Protheus, se habilitada
├── frontend/                   # Chat público React/Vite
├── middleware/                 # Proxy Node/Express
├── docker-compose.yml
└── PROJECT_MEMORY.md           # Documento histórico de arquitetura
```

3.1 Arquivos/serviços relevantes para a integração atual

Os nomes abaixo podem variar no repositório; antes de alterar, localizar por busca:

- `backend/app/services/protheus_queryrest_client.py`
- `backend/app/services/protheus_token_service.py`

- `backend/app/services/protheus_service.py`
- `backend/app/services/sync_dictionary_v52.py`
- `backend/app/api/admin_routes.py`
- `backend/app/api/tenant_routes.py`
- `backend/app/models/knowledge.py`
- `backend/app/models/catalog_v52.py`
- `backend/app/core/security.py`
- `admin-frontend/src/**` contendo formulários de Tenant/Empresa e chamadas `sync-modules/sync-schema`

Comandos úteis:

```
grep -RIn --exclude-dir=.git --exclude-dir=node_modules \
  -E 'QueryRest|oauth2|token|protheus_rest_connections|sync-modules|sync-schema' \
  backend admin-frontend middleware

find backend -type f \( -iname '*protheus*' -o -iname '*queryrest*' -o -iname '*toke
```

4. Modelos de contexto e isolamento

4.1 Identificadores obrigatórios

Toda operação que acessa a conexão Protheus deve possuir, de forma explícita:

```
tenant_code
environment_code
```

Para consultas de dados, normalmente também serão necessários:

```
company_code e/ou company_id
branch / filial
module_code
usuário e papel do Copilot
```

4.2 Regra crítica de ambiente

Não existe fallback automático aceitável para `default` quando o tenant possui um ambiente real explícito.

Proibido:

```
environment_code = environment_code or "default"
```

Proibido:

```
WHERE environment_code IN (:environment_code, 'default')
```

Obrigatório:

```
if not environment_code or str(environment_code).strip().lower() in {"none", "defaul
    raise ValueError("environment_code Protheus é obrigatório e deve ser real.")
```

Exemplo real de contexto observado:

```
tenant_code = rodol_prod
environment_code = c8te0u_prod
```

O ambiente deve ser preservado no frontend, no request Pydantic, na rota, no serviço de conexão, no token cache e no cliente QueryRest.

4.3 Schema PostgreSQL de tenant

O modelo arquitetural histórico prevê um schema por tenant, por exemplo:

```
public                  # governança global
"rodol_prod"            # dados de governança/metadados isolados do tenant
```

O nome do schema é identificador SQL e deve passar por whitelist. Nunca interpolar livremente um valor recebido do cliente em SQL.

Exemplo seguro de validação:

```
_SCHEMA_PATTERN = re.compile(r"^[a-z][a-z0-9_]{0,62}$")
_RESERVED_SCHEMAS = {"public", "pg_catalog", "information_schema"}


def quote_tenant_schema(schema_name: str) -> str:
    value = str(schema_name or "").strip().lower()
    if not _SCHEMA_PATTERN.fullmatch(value) or value in _RESERVED_SCHEMAS:
        raise ValueError("schema de tenant inválido")
    return f'"{value}"'
```

5. Banco de dados

5.1 Separação de responsabilidades

Objeto	Escopo	Responsabilidade	Não pode conter
`public.protheus_modules_master`	Global	Catálogo de módulos Protheus	tenant, empresa, URL REST, usuário, senha, token
`public.protheus_rest_connections`	Tenant + ambiente	Conexão REST e tokens cifrados	dados de catálogo global
`public.tenant` ou equivalente	Global/plataforma	Cadastro e status de tenant	fonte operacional de credencial REST após a migração
`{tenant}.company_info`	Tenant	Empresas/filiais e dados cadastrais	credenciais REST após a migração
`{tenant}.tenant_schemas`	Tenant	Catálogo/dicionário autorizado por tenant	token e senha
`{tenant}.dictionary_*`	Tenant	Snapshot detalhado de tabelas/campos/índices	token e senha
`tenant_table_permissions`	Tenant/governança	Permissões de tabela	dados de credenciais
`tenant_field_permissions`	Tenant/governança	Permissões e mascaramento de campo	dados de credenciais

5.2 Catálogo global de módulos

Tabela-alvo:

```
public.protheus_modules_master
```

Fonte de verdade:

```
SELECT DISTINCT
    USR_MODULO,
    USR_CODMOD
FROM SYS_USR_MODULE
WHERE D_E_L_E_T_ <> '*'
ORDER BY USR_MODULO;
```

Regra obrigatória:

```
mod_code  = USR_MODULO
mod_sigla = USR_CODMOD (ex.: SIGAFAT, SIGAFIN, SIGACOM)
```

A sigla serve para UI, busca e apresentação. O relacionamento entre catálogo global, permissões e dicionário deve usar `mod_code`.

Modelo mínimo:

```
CREATE TABLE IF NOT EXISTS public.protheus_modules_master (
    mod_code            INTEGER PRIMARY KEY,
    mod_sigla           VARCHAR(60) NOT NULL,
    mod_name            VARCHAR(160),
    active              BOOLEAN NOT NULL DEFAULT TRUE,
    source_updated_at   TIMESTAMPTZ,
    synced_at           TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

5.3 Conexões REST por tenant e ambiente

Tabela-alvo:

```
public.protheus_rest_connections
```

Contrato mínimo:

```
CREATE TABLE IF NOT EXISTS public.protheus_rest_connections (
    id                              BIGSERIAL PRIMARY KEY,
    tenant_code                     VARCHAR(100) NOT NULL,
    environment_code                VARCHAR(100) NOT NULL,
    base_rest_url                   VARCHAR(500) NOT NULL,
    auth_mode                       VARCHAR(30) NOT NULL DEFAULT 'oauth2_password',
    protheus_username               VARCHAR(255) NOT NULL,
    encrypted_protheus_password     TEXT NOT NULL,
    encrypted_access_token          TEXT,
    encrypted_refresh_token         TEXT,
    access_token_expires_at         TIMESTAMPTZ,
    token_updated_at                TIMESTAMPTZ,
    last_auth_error                 TEXT,
    last_auth_status                INTEGER,
    last_success_at                 TIMESTAMPTZ,
    active                          BOOLEAN NOT NULL DEFAULT TRUE,
    created_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at                      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_protheus_rest_connections_tenant_environment
        UNIQUE (tenant_code, environment_code),
    CONSTRAINT ck_protheus_rest_connections_auth_mode
        CHECK (auth_mode IN ('oauth2_password'))
);
```

Regras:

- Uma conexão ativa por par `tenant_code + environment_code`.
- URL base deve terminar em `/rest`, mas não deve incluir `/QueryRest` nem `/api/oauth2/v1/token`.

- `auth_mode` técnico padronizado: `oauth2_password`.
- Tokens e senha devem ser cifrados com o mecanismo oficial da aplicação.
- Nunca gravar/retornar tokens em texto claro.
- Não criar `environment_code = default` automaticamente.

Consulta segura de diagnóstico, sem expor segredos:

```
SELECT
    tenant_code,
    environment_code,
    base_rest_url,
    auth_mode,
    protheus_username,
    active,
    access_token_expires_at,
    token_updated_at,
    last_auth_status,
    last_auth_error,
    last_success_at
FROM public.protheus_rest_connections
WHERE tenant_code = :tenant_code
ORDER BY environment_code;
```

5.4 Dicionário e governança

Documentação histórica e implementação podem divergir quanto a nomes físicos, mas os conceitos são obrigatórios:

- `tenant_schemas`: relação permitida de tabelas/módulos por tenant.
- `dictionary_tables`: tabelas do Protheus, código de tabela, módulo e descrição.
- `dictionary_fields`: campos, tipos, tamanhos, títulos e regras.
- `dictionary_indexes` e `dictionary_fields_index`: índices e composição.
- `field_rules`: semântica específica por tenant.
- `tenant_table_permissions`: `can_list`, `can_describe`, `can_query`.
- `tenant_field_permissions`: `can_select`, `can_filter`, `masked_flag`.

Campos `can_insert` e `field_mode` não fazem parte do contrato de governança atual.

6. Integração REST Protheus

6.1 Objetivo

Utilizar OAuth2/JWT para obter um token de acesso e chamar APIs REST do Protheus, incluindo `QueryRest`, sem reenviar usuário e senha em cada consulta.

6.2 Endpoints

Considerando `base_rest_url`:

```
https://HOST:PORTA/rest
```

Endpoints:

```
POST {base_rest_url}/api/oauth2/v1/token
POST {base_rest_url}/QueryRest
```

6.3 Fluxo de token

```
sequenceDiagram
    participant Admin as Admin Frontend
    participant API as Backend
    participant DB as PostgreSQL
    participant ERP as Protheus REST

    Admin->>API: PUT Tenant + connection
    API->>ERP: POST /api/oauth2/v1/token
    ERP-->>API: access_token, refresh_token, expires_in
    API->>DB: UPSERT conexão e tokens cifrados
    API-->>Admin: conexão validada

    API->>DB: lê conexão tenant+ambiente
    API->>ERP: POST /QueryRest + Authorization Bearer
    ERP-->>API: resultado da consulta
```

6.4 Obtenção e renovação

Comportamento exigido:

1. Se existir `access_token` válido com margem de segurança de 5 minutos, reutilizar.
2. Se estiver vencido/próximo do vencimento, tentar `refresh_token`.
3. Se refresh falhar, usar `grant_type=password` com a senha REST cifrada.
4. Cifrar e persistir tokens retornados.
5. Usar lock por `tenant_code + environment_code` para impedir login concorrente.
6. Em HTTP 401 do QueryRest, invalidar o token, renovar uma única vez e repetir a chamada somente uma vez.
7. Em segundo 401, falhar com erro controlado.
8. Em HTTP 403, não repetir token indefinidamente; reportar falta de autorização do usuário/token.

6.5 Requisição de token

A forma exata aceita deve ser validada na instalação Protheus. Preferir corpo `application/x-www-form-urlencoded`, evitando senha na URL:

```
async with httpx.AsyncClient(timeout=60, follow_redirects=False) as client:
    response = await client.post(
        f"{base_rest_url.rstrip('/')}/api/oauth2/v1/token",
        headers={
            "Accept": "application/json",
            "Content-Type": "application/x-www-form-urlencoded",
        },
        data={
            "grant_type": "password",
            "username": protheus_username,
            "password": protheus_password,
        },
    )
```

Refresh esperado:

```
response = await client.post(
    f"{base_rest_url.rstrip('/')}/api/oauth2/v1/token",
    headers={
        "Accept": "application/json",
        "Content-Type": "application/x-www-form-urlencoded",
    },
    data={
        "grant_type": "refresh_token",
        "refresh_token": refresh_token,
    },
)
```

O código deve validar:

```
if response.status_code >= 400:
    raise ProtheusAuthError(...)

data = response.json()
if not data.get("access_token"):
    raise ProtheusAuthError("Resposta OAuth2 sem access_token", ...)
```

6.6 Chamada QueryRest

Obrigatório:

```
headers = {
    "Accept": "application/json",
    "Content-Type": "application/json",
    "Authorization": f"Bearer {access_token}",
}
```

```
response = await client.post(
    f"{base_rest_url.rstrip('/')}/QueryRest",
    headers=headers,
    json={"query": oracle_sql},
)
```

Proibido em QueryRest após a migração OAuth2:

```
auth=(username, password)
```

```
Authorization: Basic ...
```

```
client.post(queryrest_url, json=payload)  # sem Authorization
```

6.7 Diagnóstico HTTP

Status	Significado	Ação
200	Sucesso	Processar resultado
400 no token	Parâmetros/formato/credencial ou OAuth2 não aceito	Preservar erro sanitizado; validar request e AppServer
401 no QueryRest	Token ausente, inválido ou expirado	Renovar uma única vez e repetir
403 no QueryRest	Token reconhecido, mas usuário sem autorização	Revisar permissões REST/QueryRest; não repetir indefinidamente
404	Endpoint/URI incorreto ou serviço não publicado	Validar URL e AppServer
5xx	Falha Protheus/transporte	Reportar como indisponibilidade; aplicar timeout/retry controlado

6.8 Logs seguros

Permitido:

```
logger.info(
    "querytest_auth_context tenant=%s environment=%s header_present=%s scheme=%s toke
    tenant_code,
    environment_code,
    bool(headers.get("Authorization")),
    headers.get("Authorization", "").split(" ", 1)[0] if headers.get("Authorization")
    len(access_token),
)
```

Proibido:

```
logger.info("token=%s", access_token)
logger.info("headers=%s", headers)
logger.info("password=%s", password)
logger.info("payload=%s", payload)
```

7. Cadastro e atualização de tenant

7.1 Regra alvo

O cadastro de tenant não é a fonte de credenciais REST. O formulário pode exibir o bloco visual **Conexão Protheus**, mas os dados desse bloco devem ser gravados somente em `public.protheus_rest_connections`.

A tabela de tenant contém somente dados cadastrais, de operação, prompt, plano e status. `company_info` contém dados de empresa/filial. Nenhuma das duas deve ser usada como fallback operacional para URL, usuário, senha ou token REST após a migração.

7.2 Contrato API recomendado

```
{
  "tenant_code": "rodol_prod",
  "name": "Rodol Ltda",
  "cnpj": "02.524.305/0001-28",
  "status": "active",
  "plan_code": "pro",
  "licenca_uso": null,
  "system_prompt": "...",
  "temperature": 0.2,
  "connection": {
    "environment_code": "c8te0u_prod",
    "base_rest_url": "https://host:10707/rest",
    "auth_mode": "oauth2_password",
    "protheus_username": "USUARIO_TECNICO"
  }
}
```

Quando a senha REST for definida ou alterada:

```
{
  "connection": {
    "protheus_password": "SENHA_REST_PROTHEUS"
  }
}
```

7.3 Nunca enviar

Não aceitar nem enviar na raiz do payload de tenant:

```
protheus_rest_url
protheus_user
protheus_password
auth_mode
password
admin_password
```

Esses dados pertencem a `connection.*` quando forem credenciais REST. A senha de login do painel não pertence ao payload de tenant sob nenhuma hipótese.

7.4 Regras de senha

- A senha exibida/armazenada para login do painel é totalmente distinta da senha REST Protheus.
- `protheus_rest_password` é estado próprio do frontend.
- Não usar variável genérica `password` para mais de um contexto.
- Se edição não altera a senha REST, omitir `connection.protheus_password`; não enviar string vazia.
- Se URL, usuário, ambiente ou modo OAuth mudarem, exigir nova senha REST e novo token.
- A resposta da API nunca inclui senha, token ou valor cifrado.

7.5 Persistência transacional

Fluxo:

1. Validar payload sem logar segredo.
2. Validar `environment_code` real.
3. Se a conexão for nova/alterada e possuir senha, solicitar token antes de confirmar a conexão.
4. Cifrar senha/tokens.
5. Salvar tenant cadastral.
6. Fazer UPSERT na conexão pelo par tenant+ambiente.
7. Commit de dados internos.
8. Retornar status seguro de validação.

Se a política exigir conexão válida para tenant ativo, uma falha OAuth2 impede ativação/atualização da conexão. O erro para frontend deve ser explícito, por exemplo `502 Falha ao autenticar no REST Protheus`, não 500 genérico.

8. Sincronização de módulos e dicionário

8.1 Fluxo de módulos

1. Frontend envia `tenant_code` e `environment_code`.
2. Backend carrega exatamente a conexão ativa do par.
3. Backend obtém token válido.
4. Backend consulta `SYS_USR_MODULE` via QueryRest.
5. Backend atualiza `public.protheus_modules_master` por `mod_code`.
6. Frontend recebe lista de siglas para seleção.

Consulta de referência:

```
SELECT DISTINCT
    USR_MODULO AS mod_code,
    TRIM(USR_CODMOD) AS mod_sigla
FROM SYS_USR_MODULE
WHERE D_E_L_E_T_ <> '*'
ORDER BY USR_MODULO
```

8.2 Fluxo de dicionário

1. Usuário seleciona módulos.
2. Frontend envia tenant, ambiente e módulos.
3. Backend resolve as siglas selecionadas para `mod_code` no catálogo global.
4. Backend garante que os códigos estão autorizados no tenant.
5. Backend consulta SX2/SX3/SIX via QueryRest.
6. Backend grava no schema do tenant/tabelas de dicionário.
7. Backend aplica metadados de módulo, empresa/filial e permissões.

8.3 Contratos mínimos de sincronização

```
{
  "tenant_code": "rodol_prod",
  "environment_code": "c8te0u_prod",
  "modules": ["SIGACOM", "SIGAFAT"]
}
```

Em todas as rotas, `environment_code` deve ser obrigatório. Nunca derivar automaticamente de `default`, `None`, empresa ou `.env`.

8.4 Problemas já observados

Sintoma	Causa provável	Correção
`conexão ... rodol_prod/default`	fallback silencioso para default	exigir ambiente no frontend/backend
`conexão ... rodol_prod/None`	campo não enviado ou nomes divergentes	padronizar `environment_code`
401 QueryRest	token/credencial ausente ou inválida	obter/renovar OAuth2 e usar Bearer
403 QueryRest	autorização insuficiente ou header ausente	confirmar Bearer e permissões Protheus
400 token OAuth2	formato da chamada ou credencial inválida	validar form/query, usuário/senha e AppServer
500 no painel para erro externo	mapeamento incorreto de exceção	converter em 4xx/502/504 adequados

9. Regras de SQL Protheus/Oracle

9.1 Regras gerais

- O banco operacional alvo é Oracle.
- Todo `SELECT` em tabela Protheus deve conter o filtro de deleção lógica:

```
D_E_L_E_T_ <> '*'
```

- Não usar `TOP` nem `LIMIT`.
- Limite simples:

```
WHERE ROWNUM <= 500
```

- Paginação Oracle 12c+:

```
OFFSET 0 ROWS FETCH NEXT 100 ROWS ONLY
```

- SQL gerado pela IA passa por Query Guard e somente pode acessar tabelas/campos autorizados.

9.2 Join de faturamento

```
SF2.F2_FILIAL = SD2.D2_FILIAL
AND SF2.F2_DOC = SD2.D2_DOC
AND SF2.F2_SERIE = SD2.D2_SERIE
```

TES:

```
SD2.D2_TES = SF4.F4_CODIGO
```

Para notas normais, quando aplicável:

```
SF2.F2_TIPO = 'N'
```

Para duplicação financeira, quando aplicável:

```
SF4.F4_DUPLIC = 'S'
```

9.3 Join de compras/recebimento

```
SF1.F1_FILIAL = SD1.D1_FILIAL
AND SF1.F1_DOC = SD1.D1_DOC
AND SF1.F1_SERIE = SD1.D1_SERIE
AND SF1.F1_FORNECE = SD1.D1_FORNECE
AND SF1.F1_LOJA = SD1.D1_LOJA
```

TES:

```
SD1.D1_TES = SF4.F4_CODIGO
```

9.4 Query Guard

A implementação deve impedir:

- comandos DML/DDL em QueryRest analítico;
- acesso a tabela não autorizada;
- acesso a campo sem `can_select`;
- filtro em campo sem `can_filter`;
- vazamento de campo `masked_flag`;
- ausência de filtro lógico `D_E_L_E_T_ <> '*'` em tabelas Protheus aplicáveis;
- cross-tenant;
- SQL não Oracle.

10. RAG, LLM e comportamento do agente

10.1 Pipeline esperado

```
Pergunta do usuário
→ identificação de tenant, empresa, ambiente, perfil e módulo
→ busca de tabelas/campos/regras permitidos no catálogo tenant
→ montagem de contexto controlado
→ LLM gera plano/SQL Oracle
→ Query Guard valida
→ QueryRest executa
→ mascaramento/formatador de resposta
→ tabela Markdown e explicação
→ auditoria da consulta
```

10.2 Proibição de alucinação

O agente deve reportar exatamente condições como:

- ERP inacessível;
- token inválido;
- usuário sem permissão;
- nenhuma tabela autorizada;
- consulta sem registros;
- timeout;
- módulo não habilitado;
- empresa/filial não localizada.

Nunca preencher resultados com exemplos fictícios apresentados como dados reais.

10.3 Modelos LLM

A plataforma pode alternar entre:

- Gemini;
- OpenAI;
- Ollama local.

Variáveis típicas:

```
LLM_BACKEND
a GEMINI_API_KEY
GEMINI_MODEL
OLLAMA_BASE_URL
OLLAMA_MODEL
```

O mecanismo selecionado não pode receber dados fora das permissões resolvidas para o contexto atual.

11. Segurança

11.1 Segredos

Segredos incluem:

- senha de painel;
- senha REST Protheus;
- access token;
- refresh token;
- JWT Secret/Fernet key;
- chaves R2;
- chaves de provedores LLM;
- tokens Hetzner/Cloudflare.

Regras:

- Não registrar em log.
- Não colocar em Markdown, issues, PRs, commits ou documentação compartilhada.
- Não retornar por API GET/PUT/PATCH.
- Não reutilizar senha de painel como senha Protheus.
- Não usar senha ou token como parte de URL em produção.
- Cifrar no banco com o mecanismo central da aplicação.

11.2 Autorização do painel

- API de painel usa autenticação JWT própria do Copilot.
- O token JWT do painel não é o token OAuth2 do Protheus.
- A rota Adminer é exclusiva de `role=admin`.
- Não remover dependências de autenticação administrativa.

11.3 CORS, Cloudflare e prefetch

`503 Service Unavailable (from prefetch cache)` pode ser ruído de prefetch/Speed Brain da Cloudflare/Chrome e não erro da API de negócio. Para painel administrativo, considerar desabilitar Speed Brain no subdomínio/rota administrativa se estiver causando ruído. Diagnosticar sempre a requisição real `POST`/`PUT`/`PATCH`, não apenas a requisição especulativa.

11.4 Auditoria

Registrar:

- usuário Copilot;
- tenant, ambiente, empresa/filial;
- recurso acessado;
- ação;
- duração;
- status;
- identificador de correlação.

Não registrar payload contendo segredos ou dados além do necessário.

12. APIs e contratos

12.1 Princípios

- Requests Pydantic devem usar contratos explícitos.
- Durante migrações, suporte formato legado somente de forma temporária e converta internamente.
- Após migração, `extra='forbid'` nos modelos críticos para evitar campos secretos/legados inesperados.
- Erros de integração externa devem preservar semântica HTTP.

12.2 Status recomendados

Caso	Status
Request inválido / campo obrigatório	400 ou 422
Tenant/conexão não encontrada	404
Token do painel inválido	401
Usuário do painel sem permissão	403
Token Protheus inválido/expirado depois de retry	401 ou 502 conforme contrato
Usuário Protheus sem QueryRest	403
Falha no token OAuth2 do Protheus	502
Timeout Protheus	504
Falha interna de persistência	500

Nunca transformar um 403/400 de Protheus em 500 genérico sem contexto.

12.3 Tenant update — esquema alvo

```
class ProtheusConnectionUpdate(BaseModel):
    environment_code: str
    base_rest_url: str
    auth_mode: Literal["oauth2_password"]
    protheus_username: str
    protheus_password: str | None = None

class TenantUpdateRequest(BaseModel):
    tenant_code: str
    name: str
    cnpj: str | None = None
    status: str = "active"
    plan_code: str | None = None
    licenca_uso: str | None = None
    system_prompt: str | None = None
    temperature: float | None = 0.2
    connection: ProtheusConnectionUpdate
```

13. Estado atual e divergências conhecidas

13.1 Documentação histórica versus alvo atual

A documentação V5 anterior citava credenciais REST em `tenant` e/ou `{tenant}.company_info`, com fallback por conectores e `.env`. Esse modelo é **legado** para a integração atual.

A decisão atual é:

```
Credenciais REST e tokens → public.protheus_rest_connections
Chave de resolução        → tenant_code + environment_code
Catálogo de módulos       → public.protheus_modules_master (global)
Dados cadastrais de tenant  → tabela de tenant
Dados empresa/filial      → company_info
```

Durante a refatoração, remover gradualmente:

- leituras de `tenant_apirest_url`, `tenant_protheus_user`, `tenant_encrypted_protheus_password`;
- leituras de URL/usuário/senha em `company_info`;
- fallback para `.env` em fluxos de tenant real;
- fallback para ambiente `default`.

13.2 Incidentes observados em 19/08/2026

1. Coluna `tenant_name` inexistente no banco real; o código deveria usar os nomes reais da tabela implantada, como `tenant_code`, `tenant_status` e outros campos com prefixo `tenant_`.
2. A sincronização inicialmente retornou 401 no QueryRest, depois 403.
3. O cadastro/integração tentou criar conexão indevida para `environment_code=default`.
4. Rotas diferentes tentaram resolver o ambiente como `default` e como `None`.
5. Frontend enviou campos legados de conexão na raiz do tenant.
6. Frontend misturou senha administrativa do painel com `protheus_password`.
7. A chamada ao token `/rest/api/oauth2/v1/token?grant_type=password` retornou 400 no Protheus.
8. `QueryRest` retornou 403 mesmo com contexto tenant/ambiente correto, indicando necessidade de verificar Bearer e/ou permissão do usuário técnico.

13.3 Prioridade operacional atual

1. Corrigir cadastro de tenant para contrato `connection.*`.
2. Corrigir emissão de token OAuth2 — formato, senha REST real, tratamento de erro e persistência.
3. Confirmar que QueryRest envia Bearer em todas as saídas HTTP.
4. Confirmar permissões do usuário técnico Protheus para QueryRest/SX2/SX3/SIX.
5. Somente depois executar sincronização de módulos e dicionário.

14. Estratégia de testes

14.1 Unitários

Cobrir:

- normalização de `tenant_code` e `environment_code`;
- bloqueio de `default`/`None` como ambiente implícito;
- validação de URL base REST;
- normalização de módulo (`SIGAFAT`, `FAT`, código numérico);
- validade de token com margem de segurança;
- escolha refresh versus password grant;
- mascaramento de logs;
- construção de header Bearer;
- ausência de senha no response schema;
- rejeição de campos legados no payload final.

14.2 Integração

- Criar/editar tenant com conexão nova.
- Editar apenas dados cadastrais sem enviar senha REST.
- Alterar URL/usuário/ambiente exigindo senha REST.
- Token 200 e persistência cifrada.
- QueryRest com Bearer.
- Retry único após 401.
- 403 preservado para UI.
- Sincronização de módulos com ambiente explícito.
- Sincronização de dicionário com módulos autorizados.
- Isolamento de conexão por tenant e ambiente.

14.3 E2E manual

1. Login no painel.
2. Abrir Tenant.
3. Informar `environment_code` real.
4. Informar URL REST base, usuário REST e senha REST correta.
5. Salvar.
6. Verificar no Network que o payload usa `connection.*`.
7. Verificar que senha de painel não aparece no payload de tenant.
8. Validar status de conexão sem ver token/senha.
9. Sincronizar módulos.
10. Validar catálogo global de módulos.
11. Selecionar módulos e sincronizar dicionário.
12. Executar consulta pelo agente respeitando RBAC.

14.4 SQL de validação

```
SELECT
    tenant_code,
    environment_code,
    active,
    access_token_expires_at,
    last_auth_status,
    last_auth_error,
    last_success_at
FROM public.protheus_rest_connections
WHERE tenant_code = 'rodol_prod';
```

```
SELECT
    mod_code,
    mod_sigla,
    mod_name,
    active,
    synced_at
FROM public.protheus_modules_master
ORDER BY mod_code;
```

15. Operação, deploy e troubleshooting

15.1 Build/deploy

```
cd /root/copilotprotheus
git pull origin main

docker compose up -d --build backend
# ou, se necessário:
docker compose up -d --build backend admin-frontend middleware
```

`docker compose restart` não recompila imagens e não é suficiente após alteração de código.

15.2 Logs

```
docker compose logs --tail=200 -f backend
docker compose logs --tail=200 -f middleware
docker compose logs --tail=100 cloudflared
docker compose ps
```

Filtros úteis:

```
docker compose logs --tail=500 backend | grep -Ei \
  'oauth|token|queryrest|sync-modules|sync-schema|tenant|environment|401|403|400|500
```

15.3 Teste de saúde

```
curl -i http://127.0.0.1:8000/health
```

Ajustar a rota conforme a implementação real (`/health`, `/api/health` ou equivalente).

15.4 Diagnóstico de requisições do navegador

No DevTools → Network, para a requisição real, registrar de forma segura:

- URL e método;
- status;
- request payload **mascarado**;
- response body;
- headers não secretos;
- correlation/request ID quando existir.

Não copiar em tickets/chats:

- senha REST;
- token de painel;
- access token;
- refresh token;
- header Authorization.

16. Diretrizes para IAs colaboradoras

16.1 Antes de alterar qualquer código

1. Ler este documento e `PROJECT_MEMORY.md`.
2. Inspecionar a estrutura atual, sem assumir que o documento histórico representa exatamente o banco implantado.
3. Buscar referências reais no repositório antes de renomear campos/tabelas.
4. Consultar `information_schema` antes de propor DDL destrutivo.
5. Separar hipótese de evidência.
6. Não criar dados de exemplo no banco de produção.

16.2 Regras para mudanças

- Não alterar nem excluir dados/arquivos sem confirmação humana explícita.
- Não reintroduzir Basic Auth em QueryRest.
- Não reintroduzir fallback para `default`.
- Não armazenar conexão no catálogo global de módulos.
- Não usar sigla de módulo como chave de join onde `mod_code` é exigido.
- Não criar SQL Oracle sem `D_E_L_E_T_ <> '*'` em tabelas Protheus aplicáveis.
- Não expor segredo em resposta, log, teste ou documentação.

- Não mascarar erro externo como 500 interno se houver status semântico conhecido.
- Ao propor migrations, fazer aditivas e reversíveis primeiro; remoção de colunas somente depois de migração completa de leitura/escrita.

16.3 Formato de resposta esperado da IA colaboradora

Para incidentes:

1. Resumo executivo.
2. Evidências observadas.
3. Hipóteses por prioridade.
4. Validações pendentes.
5. Impactos.
6. Plano de correção com arquivos/rotas/tabelas.
7. Testes e critérios de aceite.
8. Próximos passos executáveis.

Para desenvolvimento:

1. Objetivo e escopo.
2. Contratos API.
3. Modelo de dados.
4. Fluxos.
5. Segurança.
6. Dependências.
7. Backlog técnico.
8. Plano de testes.

16.4 Prompt inicial sugerido para outra IA

```
Você está colaborando no projeto Copilot Protheus. Leia primeiro a especificação téc

Princípios obrigatórios:
- ambiente multi-tenant com isolamento rigoroso;
- zero alucinação de dados empresariais;
- Oracle/Protheus: usar D_E_L_E_T_ <> '*' quando aplicável;
- catálogo public.protheus_modules_master é global e usa mod_code = USR_MODULO;
- conexão REST usa public.protheus_rest_connections e chave tenant_code + environmen
- não usar default/None como fallback de ambiente;
- QueryRest usa Authorization Bearer, nunca Basic Auth;
- nunca registrar ou retornar senha, access token, refresh token ou Authorization;
- senha do painel não pode ser usada como senha REST Protheus;
- separar evidências de hipóteses e finalizar com testes e próximos passos.
```

17. Backlog técnico recomendado

P0 — Bloqueadores

- [ ] Padronizar o contrato de tenant para `connection.*`.
- [ ] Incluir `environment_code` obrigatório no frontend e backend.
- [ ] Remover fallback para `default` e `None`.
- [ ] Corrigir separação de senha do painel e senha REST.
- [ ] Corrigir emissão OAuth2: formato da requisição, tratamento HTTP e resposta sanitizada.
- [ ] Implementar/revisar `get_valid_access_token` por tenant+ambiente.
- [ ] Garantir header Bearer em todas as chamadas QueryRest.
- [ ] Mapear 401/403/400 Protheus para respostas API adequadas.
- [ ] Validar permissões do usuário técnico no Protheus.

P1 — Segurança e consistência

- [ ] Eliminar leituras de credencial de `tenant` e `company_info`.
- [ ] Criar testes de não vazamento de senha/token.
- [ ] Criar lock distribuído/adequado para token por tenant+ambiente se houver múltiplos workers.
- [ ] Implementar observabilidade com correlation ID, sem segredos.
- [ ] Revisar todas as chamadas HTTP do backend/middleware para garantir cliente centralizado.
- [ ] Desativar/remover conexões `default` indevidas após validação.

P2 — Governança e evolução

- [ ] Sincronização robusta de módulos por `USR_MODULO` e `USR_CODMOD`.
- [ ] Sincronização SX2/SX3/SIX por módulos selecionados.
- [ ] Completar Query Guard e RBAC de tabela/campo.
- [ ] Auditoria de consultas por tenant/usuário/contexto.
- [ ] Atualizar documentação histórica para remover contradições de credenciais legadas.

18. Checklist final antes de merge/deploy

Código

- [ ] Nenhuma chamada QueryRest sem header Bearer.
- [ ] Nenhuma chamada QueryRest com Basic Auth.
- [ ] Nenhuma senha/token em `logger`.
- [ ] Ambiente obrigatório em todas as rotas de sincronização.
- [ ] Sem fallback `default` ou `None`.
- [ ] `auth_mode` padronizado em `oauth2_password`.
- [ ] Payload tenant usa `connection.*`.
- [ ] Senha de painel e REST possuem campos/estados independentes.
- [ ] Erros externos retornam HTTP semântico.

Banco

- [ ] Uma conexão ativa por tenant+ambiente.
- [ ] Tokens e senha cifrados.
- [ ] Catálogo global não contém tenant/credencial.
- [ ] Sem conexão default indevida.
- [ ] Migrations são reversíveis/seguras.

Protheus

- [ ] Endpoint de token disponível.
- [ ] Usuário técnico dedicado e ativo.
- [ ] Permissão para token, QueryRest, SX2, SX3 e SIX.
- [ ] Segurança/OAuth2/Lib compatíveis.
- [ ] Licença/Stateless validada com sustentação TOTVS quando aplicável.

Operação

- [ ] Testes unitários e integração executados.
- [ ] Backend e admin frontend reconstruídos.
- [ ] Network sem segredo exposto.
- [ ] Logs limpos de segredos.
- [ ] Sincronização de módulos retorna 200.
- [ ] Sincronização de dicionário retorna 200.
- [ ] Consulta de agente respeita tenant, módulo, empresa, filial e RBAC.

19. Referências internas

- `PROJECT_MEMORY.md`: arquitetura histórica v5.3, topologia, regras de SQL e operação.
- Especificação funcional de arquitetura multi-tenant por schema: modelo de isolamento, dicionário e fluxo de consulta.
- Documento de autenticação REST: decisão OAuth2/Bearer, cache/refresh/retry, segurança e critérios de aceite.

Este documento deve ser atualizado sempre que houver alteração de contrato de API, estrutura de dados, fluxo de autenticação, topologia de deploy, regra de segurança ou decisão arquitetural relevante.